

CrackMe №4

Task

Author	deo
Language	C\C++
Platform	Windows
Arch	x86-64
Difficulty	3.3

Description

You need to find the key to decrypt the flag.

If you find the key, enter it and print the flag; entering the wrong key will result in a "wrong password" error.

I request the key and a detailed description to improve the packer, thank you.

Additionally, if you want to find the password, after entering any password, it is decrypted in memory for a few seconds and then wiped back. My goal here is to learn the weaknesses of the packer and improve it.

Task source

Решение

В задании сразу сказано, что использован упаковщик. Соберем всю доступную информацию о программе, чтобы понять, с чем и как именно можно работать.

Данные сканера Detect It Easy:

PE64 Property	PE64 Value
Operation System	Windows(Vista), AMD64, 64-bit, Console
Linker	Microsoft Linker(14.36.35226)
Compiler	Microsoft Visual C/C++ [C++]
Language	C++
Tool	Visual Studio 2022
(Heur) Protection	Generic: No IAT + nop EP padding
(Heur) Packer	Section 5 (".elpk") compressed

Посмотрим карту памяти и секции в том же детекторе:

Смещение	Адрес	Размер	Имя		Entropy	Status
0x0	0x0	0x400	PE Header	Normal Section	2.84298	Not Packed

Смещение	Адрес	Размер	Имя		Entropy	Status
0x400	0x1000	0x16800	'.text' Section	Normal Section	0.00002	Not Packed
0x1c00	0x18000	0xa800	'.rdata' Section	Normal Section	0.00003	Not Packed
0x21400	0x23000	0xc00	'.data' Section	Normal Section	0.00047	Not Packed
0x22000	0x25000	0x1400	'.pdata' Section	Normal Section	5.01671	Not Packed
0x23400	0x27000	0x800	'.reloc' Section	Normal Section	4.93691	Not Packed
0x23c00	0x28000	0x1f800	'.elpk' Section	Unusual, Probably Packed-related	7.99862	Packed
0x43400	0x48000	0x4600	'.elst' Section	Unusual, Probably Packed-related	6.64244	Packed
0x47a00	0x4d000	0x5800	'.eldat' Section	Unusual, Probably Packed-related	7.96867	Packed

Посмотрим бинарь в pestudio

Characteristics	Value	Value	Value	Value	Value	Value	Value	Value
Section Name	.text	.rdata	.data	.pdata	.reloc	.elpk	.elst	.eldat
Write	-	-	X	-	-	X	X	-
Execute	X	-	-	-	-	-	X	-
Share	-	-	-	-	-	-	-	-
Self-modifying	-	-	-	-	-	-	X	-
Virtual	-	-	-	-	-	-	-	-
Directory->Exception	-	-	-	0x25000	-	-	-	-
Base-Of-Code	0x1000	-	-	-	-	-	-	-
Entry-Point->location	-	-	-	-	-	-	0x48000	-

Обобщаем:

- **.elst :**
Для RWX нужны функции типа VirtualProtect, NtProtectVirtualMemory, VirtualAlloc, WriteProcessMemory - ловим вызовы по брейкпоинтам, ищем переход на ОЕР и снимаем дампы?
- **Import/Exports/Library:**
В pestudio оно всё п/а. Следовательно, не существуют до распаковки. Может быть, будут JIT/RT-вызовы через GetProcAddress и LoadLibrary - если поломается IAT, отловить вызовы и починить?
- **Какая секция и для чего?**
 - **.elpk:** -W-. Скорее всего пейлоад?
 - **.elst:** RWX. Скорее всего распаковщик?
 - **.eldat:** ---. Возможно системная информация, или IAT?

Посмотрим в IDA, есть ли что-то интересное в .elst:

```
.elst:000000014004BA20      push    rcx
.elst:000000014004BA21      pop     rcx
.elst:000000014004BA22      call    near ptr 13F5D66AEh
.elst:000000014004BA27      add     rax, 0
-----
.elst:000000014004BCB6      push    rbp
.elst:000000014004BCB7      mov     rbp, rsp
.elst:000000014004BCBA      sub     rsp, 20h
.elst:000000014004BCBE      jmp     near ptr 0EEACA3B3h
```

Инструкции по адресам 0x4ba22 и 0x4bcbe имеют странное константное значение. Возможно, эти адреса/константы станут доступны/превратятся в нормальный код после распаковки. Возможно, по ним удастся выйти на OEP.

Переходим к работе с отладчиком. При запуске через x64dbg посмотрим на карту памяти, сразу после инициализации процесса:

Секция	До распаковки	После распаковки
.elrk	Адрес=0000000140028000 Размер=0000000000020000 Группа=Пользователь Информация о странице=".elrk" Тип выделения=IMG Текущие права доступа=-RWC- Защита при выделении=ERWC-	Адрес=0000000140028000 Размер=0000000000020000 Группа=Пользователь Информация о странице=".elrk" Тип выделения=IMG Текущие права доступа=---- Защита при выделении=ERWC-
.elst	Адрес=0000000140048000 Размер=0000000000005000 Группа=Пользователь Информация о странице=".elst" Тип выделения=IMG Текущие права доступа=ERWC- Защита при выделении=ERWC-	Адрес=0000000140048000 Размер=0000000000005000 Группа=Пользователь Информация о странице=".elst" Тип выделения=IMG Текущие права доступа=ERWC- Защита при выделении=ERWC-
.eldat	Адрес=000000014004D000 Размер=0000000000006000 Группа=Пользователь Информация о странице=".eldat" Тип выделения=IMG Текущие права доступа=-R--- Защита при выделении=ERWC-	Адрес=000000014004D000 Размер=0000000000006000 Группа=Пользователь Информация о странице=".eldat" Тип выделения=IMG Текущие права доступа=-R--- Защита при выделении=ERWC-

Получается, что секция .elrk могла использоваться как контейнер, после выполнения кода была закрыта и исполнение оттуда не происходит.

.elst остается ERW и до, и после - распакованный код остается внутри. Значит, надо следить за рантайм-изменением этой секции в памяти.

По итогу решил посмотреть, где будет храниться пользовательский ввод и что можно будет найти в памяти после обработки ввода. После вывода на экран сообщения "wrong password" пользовательский ввод находится в стеке. Сдампим стек, изучим:

78 39 4B 21 37 7A 50 71 34 6D 4E 76 52 32 62 4C -> x9K!7zPq4mNvR2bL

65 78 70 61 6E 64 20 33 32 2D 62 79 74 65 20 6B -> expand 32-byte k
70 61 73 73 77 6F 72 64 -> password

Строка "expand 32-byte k" показалась интересной. Оказалось, что использовалось шифрование ChaCha20/Salsa20-family

Проверим, не пароль ли это:

```
=== elevenpack crackme ===  
Enter password: x9K!7zPq4mNvR2bL  
[+] FLAG{p3Wq8RnZmK4vYxT7Bc2J}
```

А вот теперь интересно - получится ли достать ключ?

Отсюда можно узнать структуру ChaCha, особенно инициализации состояния. Попробуем найти эту информацию в дампе:

```
65 78 70 61 6E 64 20 33 32 2D 62 79 74 65 20 6B ///"expand 32-byte k" magic constant, 4 words  
9F 42 B7 1E 88 C3 05 77 2A E9 10 D6 4B 33 6C A1 /// 8 words from the 256-bit key  
58 0F E2 95 71 44 BE 29 CD 83 16 6F B0 07 D8 52  
01 00 00 00 /// 1 word for block counter  
11 22 33 44 55 66 77 88 99 AA BB CC /// 3 words for the 96-bit nonce
```